

SUPERSTORE SALES & PROFITABILITY ANALYSIS

Exploratory Data Analysis & KPI Dashboard — Project Documentation

Prepared by:	Ahmed Taher Mohamed Sobhy Ghanem
Tools used:	Python (pandas, numpy, matplotlib, seaborn)
Dataset:	Sample Superstore — Orders, Returns & People (File 1.xls)
Project Notebook:	https://colab.research.google.com/drive/1N-7hA1hQQhAdPy65K-FxZyJJYAcXCzGw?usp=sharing
Report date:	August 2026

Table of Contents

1.	Introduction	3
2.	Methodology	4
3.	Key Performance Indicators (KPIs)	6
4.	Visualizations & Insights	8
5.	Key Findings & Recommendations	15
6.	Limitations & Possible Next Steps	16

1. Introduction

This report walks through a small end-to-end data analysis project built on the Sample Superstore dataset — a retail transactions dataset that most data analysts will recognize, since it is one of the most commonly used datasets for practicing business intelligence and exploratory data analysis. It contains order-level records for a fictional office-supply retailer operating across the United States between 2016 and 2019, including what was sold, how much it sold for, how much profit it generated, how it was shipped, who bought it, and whether it was later returned.

The purpose of the project was not simply to produce charts, but to work through a realistic analytics workflow from start to finish: taking a messy, multi-sheet Excel file, cleaning and consolidating it into a single usable table, engineering a handful of new features that make deeper analysis possible, calculating the KPIs a manager would actually want to see on a dashboard, and then building visualizations that answer specific business questions rather than just describing the data for its own sake. Wherever a chart raised a question, I tried to follow it with a plain-language explanation of what it means and why it matters.

The dataset itself is split across three sheets in the source workbook: **Orders** (the main transactional table, with one row per line item), **Returns** (a flag for which orders were later returned), and **People** (a lookup table mapping each sales region to its regional manager). Section 2 explains exactly how these three pieces were brought together and cleaned before any analysis began.

PROJECT CODE: The full, runnable code behind every step in this report — data cleaning, feature engineering, KPI calculations, and all eight charts — is available as a Google Colab notebook: <https://colab.research.google.com/drive/1N-7hA1hQQhAdPy65K-FxZyJJYAcXCzGw?usp=sharing>

1.1 Objectives

- Consolidate the Orders, Returns, and People sheets into a single, analysis-ready table, handling missing values and duplicate records along the way.
- Engineer a small set of derived features — shipping duration, profit margin, order month, and a sales-performance tier — that make the later visualizations possible.
- Calculate a concise set of headline KPIs that summarize overall business performance in a way a manager could read in under a minute.
- Build eight visualizations covering time trends, category and sub-category performance, correlations between key numeric variables, regional and shipping performance, the impact of discounting on profit, customer-segment comparisons, and the most and least profitable individual products.
- Turn each chart into an actual, specific business insight rather than leaving it to speak for itself, and use those insights to build a short set of practical recommendations.

2. Methodology

Before any KPI can be trusted or any chart can be believed, the underlying data has to be cleaned and put into a shape that supports analysis. This section documents exactly what was done to the raw data and, just as importantly, why each step was necessary.

2.1 Data Loading

The three sheets — **Orders**, **Returns**, and **People** — were loaded separately from the source Excel workbook using `pandas.read_excel()`, each one addressed by its sheet name. Loading them as three independent DataFrames first, rather than trying to merge them inside the spreadsheet, makes it much easier to inspect, clean, and validate each piece on its own before combining anything.

2.2 Data Cleaning & Merging

Real-world data is rarely complete, and this dataset is no exception. A few issues had to be resolved before the tables could be safely combined:

- **Missing postal codes.** A handful of orders in the Orders sheet were missing a postal code. Rather than dropping those rows and losing otherwise-valid sales data, missing values were filled with a default code (05401) and the whole column was standardized to a zero-padded, 5-digit string — since postal codes are identifiers, not numbers, and should never be treated as something to do arithmetic on.
- **Duplicate return records.** The Returns sheet contained more than one row for some order IDs. Before merging, duplicates were removed by keeping only the first occurrence of each **Order ID**, so that a single order could not accidentally get double-counted as returned twice, or inflate the return rate.
- **Combining Orders and Returns.** The cleaned Returns table was left-joined onto Orders on **Order ID**. A left join was the right choice here specifically because most orders were never returned at all, so they simply will not appear in the Returns sheet — a left join keeps every order regardless, and any order that doesn't find a match is then explicitly filled in with 'No' in the **Returned** column, rather than being left as a missing value that could later be miscounted.
- **Adding regional context.** Finally, the People sheet — which maps each region to its regional sales manager — was left-joined onto the combined table on **Region**, so that every order carries that regional context without having to look it up separately later.

```

241 import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns

251 # STEP 1: Load Data

    orders = pd.read_excel('/content/File 1.xls', sheet_name='Orders')
    returns = pd.read_excel('/content/File 1.xls', sheet_name='Returns')
    people = pd.read_excel('/content/File 1.xls', sheet_name='People')

261 # STEP 2: Clean & Merge Data

    # Fill missing postal codes with default 05401
    orders['Postal Code'] = orders['Postal Code'].fillna(5401).astype(int).astype(str).str.zfill(5)

    # Remove duplicate order returns and merge sheets
    returns_clean = returns.drop_duplicates(subset=['Order ID'])
    df = orders.merge(returns_clean, on='Order ID', how='left')
    df['Returned'] = df['Returned'].fillna("No")
    df = df.merge(people, on='Region', how='left')

271 # STEP 3: Add New Columns

    df['Shipping Duration'] = (df['Ship Date'] - df['Order Date']).dt.days
    df['Profit Margin'] = np.where(df['Sales'] > 0, df['Profit'] / df['Sales'], 0)
    df['Order YearMonth'] = df['Order Date'].dt.to_period('M')

    # Categorize Sales Performance
    q25 = df['Sales'].quantile(0.25)
    q50 = df['Sales'].quantile(0.50)
    q75 = df['Sales'].quantile(0.75)

    conditions = [
        df['Sales'] <= q25,
        (df['Sales'] > q25) & (df['Sales'] <= q50),
        (df['Sales'] > q50) & (df['Sales'] <= q75)
    ]
    choices = ['Low', 'Medium', 'High']
    df['Sales Performance Category'] = np.select(conditions, choices, default='Premium')

```

Figure 2.1 — Data loading and cleaning/merging code (Steps 1–2).

2.3 Feature Engineering

With a clean, merged table in place, four new columns were engineered specifically to support the analysis that follows. None of these exist in the raw data — they had to be derived:

- **Shipping Duration** — the number of days between Order Date and Ship Date. This turns two raw timestamps into a single number that can be averaged, compared across shipping modes, and correlated against profit (see Sections 3 and 4.5).
- **Profit Margin** — Profit divided by Sales, giving a percentage that is comparable across orders of very different sizes. Orders with zero or negative sales were explicitly set to a margin of 0 rather than left to produce a division error or a misleadingly huge ratio.
- **Order YearMonth** — the order date collapsed down to a Year-Month period (e.g. 2018-11), which is what makes it possible to group thousands of individual orders into a clean monthly trend line (Section 4.1) instead of a noisy day-by-day plot.
- **Sales Performance Category** — each order was bucketed into a Low, Medium, High, or Premium tier based on where its sales value falls relative to the 25th, 50th, and 75th percentiles of all sales. Using percentiles rather than fixed dollar thresholds means the categories adapt automatically to the actual spread of the data rather than relying on an arbitrary cutoff that might not fit the real distribution.

3. Key Performance Indicators (KPIs)

Once the data was clean, the next step was to step back and ask: if this were a real business, what handful of numbers would its owner actually want to see first? The six metrics below were chosen because together they answer that question — how much did we sell, how much did we keep, how efficiently are we operating, and how much friction (in the form of returns) is in the system.

Metric	Value
Total Sales	\$2,297,200.86
Total Profit	\$286,397.02
Profit Margin	12.47%
Total Unique Orders	5,009
Average Shipping Duration	3.96 days
Order Return Rate	5.91%

3.1 How Each KPI Was Calculated

- **Total Sales** and **Total Profit** are simply the sum of the Sales and Profit columns across every row in the cleaned dataset.
- **Profit Margin** is Total Profit divided by Total Sales, expressed as a percentage — this is the company-wide margin, distinct from the per-order Profit Margin column engineered in Section 2.3.
- **Total Unique Orders** counts distinct Order IDs rather than rows, since a single order can span several line items (several products) and would otherwise be over-counted.
- **Average Shipping Duration** is the mean of the Shipping Duration column engineered earlier.
- **Order Return Rate** is the number of unique orders flagged as returned, divided by the total number of unique orders, expressed as a percentage.

INSIGHT: A 12.47% profit margin means that, on average, roughly \$1 of every \$8 in sales is kept as profit before any further overhead is applied — a reasonable but not especially wide margin for a retail business, and one that later sections show is being actively squeezed by discounting (Section 4.3). The 5.91% return rate is not alarming on its own, but Section 4.8 shows it is not evenly spread — some sub-categories and products drive a disproportionate share of it. The 3.96-day average shipping duration sits in the middle of the four shipping modes offered, which makes more sense once you see how differently those modes perform in Section 4.5.

```
# STEP 4: Calculate & Print Results (KPIs)

total_sales = df['Sales'].sum()
total_profit = df['Profit'].sum()
margin = (total_profit / total_sales) * 100
total_orders = df['Order ID'].nunique()
avg_shipping = df['Shipping Duration'].mean()
returned_orders = df[df['Returned'] == 'Yes']['Order ID'].nunique()
return_rate = (returned_orders / total_orders) * 100

print("KEY PERFORMANCE INDICATORS")
print(f"Total Sales: ${total_sales:,.2f}")
print(f"Total Profit: ${total_profit:,.2f}")
print(f"Profit Margin: {margin:.2f}%")
print(f"Total Unique Orders: {total_orders:,}")
print(f"Avg Shipping Duration: {avg_shipping:.2f} days")
print(f"Order Return Rate: {return_rate:.2f}%")
print("-----")

KEY PERFORMANCE INDICATORS
Total Sales: $2,297,200.86
Total Profit: $286,397.02
Profit Margin: 12.47%
Total Unique Orders: 5,009
Avg Shipping Duration: 3.96 days
Order Return Rate: 5.91%
-----

# STEP 5: Create Visualizations

sns.set_theme(style="whitegrid")

# 1. Monthly Trend
plt.figure(figsize=(10, 4))
monthly = df.groupby('Order YearMonth')[['Sales', 'Profit']].sum()
plt.plot(monthly.index.astype(str), monthly['Sales'], label='Sales', color='skyblue', marker='o')
plt.plot(monthly.index.astype(str), monthly['Profit'], label='Profit', color='green', marker='s')
plt.xticks(rotation=90, fontsize=8)
plt.title("1. Monthly Sales & Profit Trend")
plt.xlabel("Month")
plt.ylabel("Amount ($)")
plt.legend()
plt.tight_layout()
plt.savefig('fig1_monthly_trend.png')
plt.show()
```

Figure 3.1 — KPI calculation code and console output.

4. Visualizations & Insights

With the KPIs establishing the big picture, the next step was to break that picture apart and look at it from different angles. Eight charts were built with matplotlib and seaborn, each one aimed at a specific question: is the business growing steadily, which products actually make money, what's driving returns and margin loss, and where should attention go first. Each chart below is followed by a plain-language read of what it shows and why that matters.

4.1 Monthly Sales & Profit Trend

INSIGHT: Sales follow a clearly seasonal, generally upward-trending pattern, with pronounced spikes toward the end of almost every year (roughly November–December), which lines up with a typical holiday and year-end purchasing cycle — likely a mix of consumer holiday shopping and corporate/office year-end budget spending. What stands out more, though, is the profit line: it stays comparatively flat and grows far more slowly than sales, and dips visibly below zero in a few specific months (notably around early 2016 and early 2019). In practice this means the business is getting bigger without becoming proportionally more profitable — a pattern that is consistent with the discounting effect uncovered later in Section 4.3, where deep discounts are shown to erode margin sharply.

4.2 Sales & Profit by Sub-Category

INSIGHT: Phones and Chairs are the two strongest sub-categories by revenue, each bringing in well over \$300,000 in total sales, with Storage and Binders not far behind. The more important detail is in the profit bars sitting next to each sales bar: **Tables** generates over \$200,000 in sales but its profit bar actually dips **below zero** — meaning the business is, in aggregate, losing money every time it sells furniture from this sub-category. Machines and Supplies show a similar (if less severe) pattern, with sales that look healthy but profit that barely registers. This makes Tables the single clearest candidate in the whole dataset for a pricing, cost, or discounting review — a conclusion reinforced again in Sections 4.4 and 4.8.

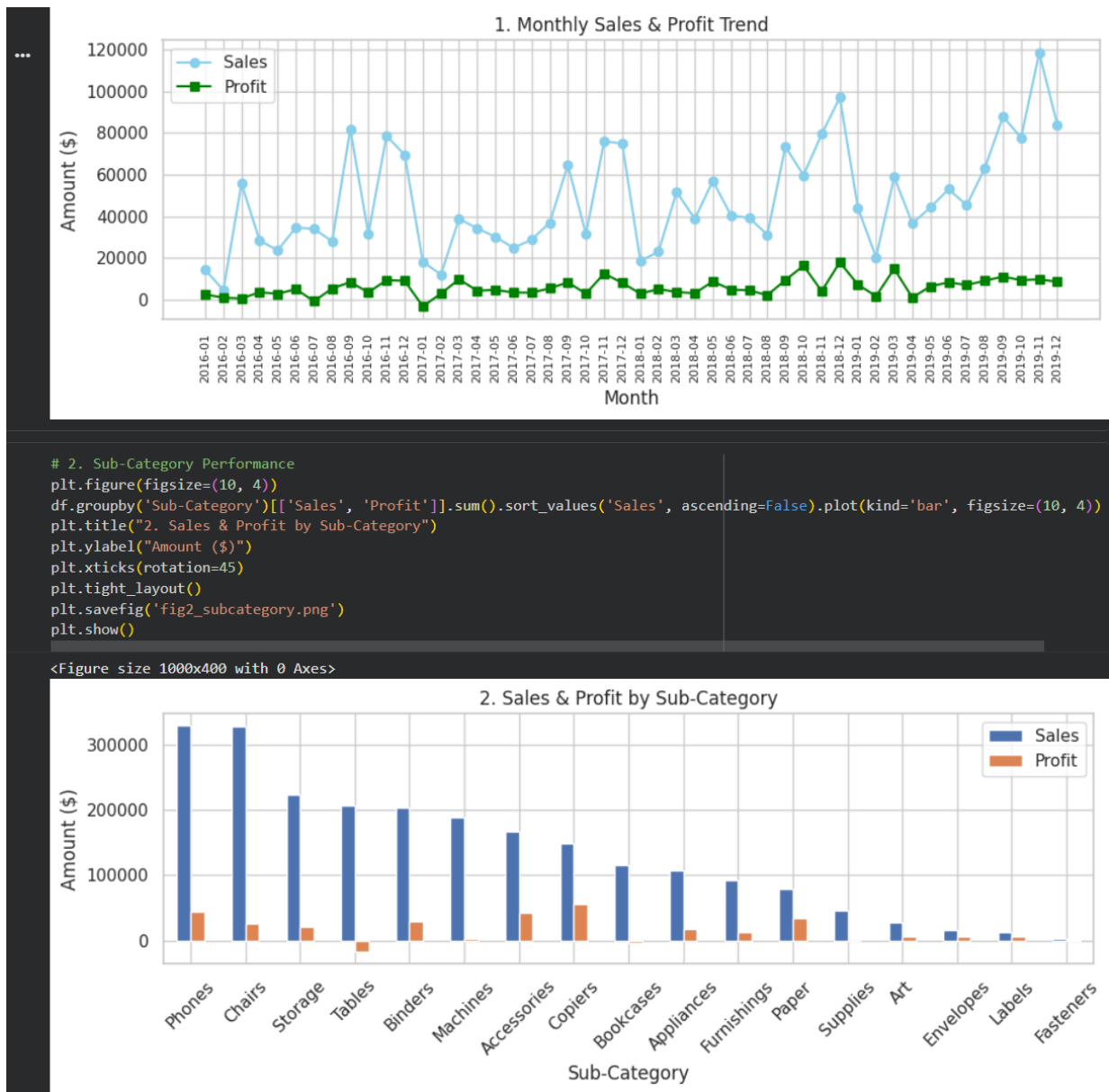


Figure 4.1 / 4.2 — Monthly trend code & chart, and sub-category performance code & chart.

4.3 Key Numeric Variable Correlations

```
# 3. Correlation Heatmap
plt.figure(figsize=(7, 5))
corr = df[['Sales', 'Quantity', 'Discount', 'Profit', 'Shipping Duration', 'Profit Margin']].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="Blues")
plt.title("3. Key Numeric Variable Correlations")
plt.tight_layout()
plt.savefig('fig3_correlation.png')
plt.show()
```

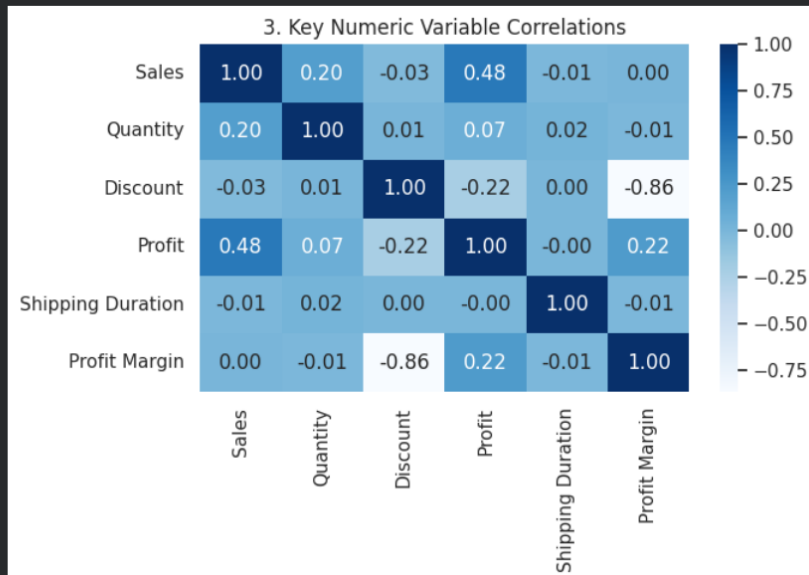


Figure 4.3 — Correlation heatmap of Sales, Quantity, Discount, Profit, Shipping Duration, and Profit Margin.

INSIGHT: This heatmap is arguably the single most useful chart in the whole report, because it points directly at what is driving profitability. Profit correlates moderately with Sales (0.48), which is unsurprising — bigger orders tend to generate more profit in raw dollar terms. The far more important number is the relationship between **Discount** and **Profit Margin**: a strong negative correlation of -0.86 . That is a very strong relationship for real-world business data, and it says plainly that as discount levels increase, profit margin collapses almost in lockstep. Discount is also negatively correlated with Profit itself (-0.22), meaning heavy discounting doesn't just shrink the margin percentage — it can and does eat into actual dollars of profit as well. By contrast, Shipping Duration shows essentially no correlation with any financial metric (all values near 0), which tells us delivery speed is not, on its own, a meaningful driver of how profitable an order turns out to be.

4.4 Profit Margin Distribution by Region

```
# 4. Regional Profit Margin
plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x='Region', y='Profit Margin', showfliers=False)
plt.title("4. Profit Margin Distribution by Region")
plt.tight_layout()
plt.savefig('fig4_region_margin.png')
plt.show()
```

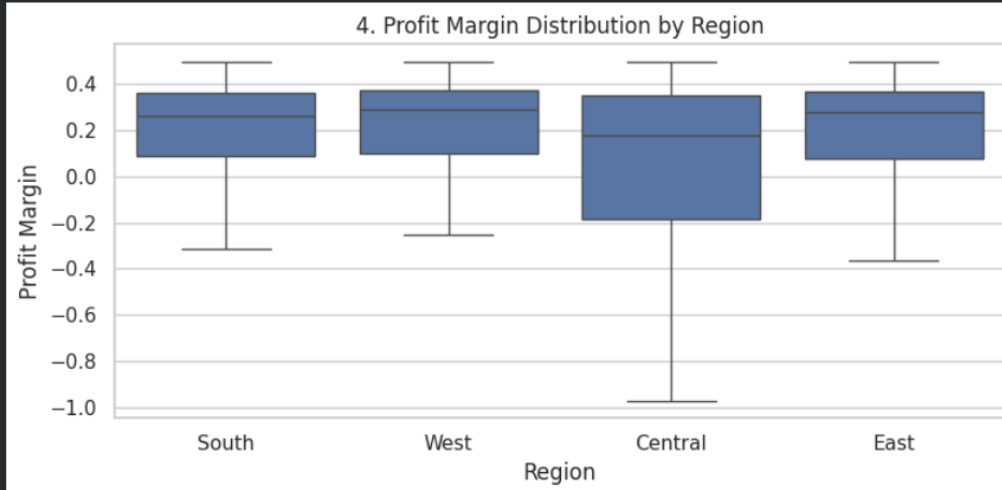


Figure 4.4 — Boxplot of profit margin by region (outliers hidden).

INSIGHT: South, West, and East all show fairly healthy, tightly-clustered profit margins, with medians sitting around 20% and a relatively narrow spread. **Central** is the clear outlier: its median margin is noticeably lower than the other three regions, and its box (representing the middle 50% of orders) is visibly wider, stretching much further down into negative territory than any other region's. In other words, Central doesn't just have a lower typical margin — it also has far more inconsistency, with a meaningfully larger share of orders actively losing money. That combination makes Central the region most worth investigating first, whether the cause turns out to be more aggressive discounting, a different product mix, or something else regional-specific.

4.5 Shipping Duration by Mode



Figure 4.5 — Boxplot of shipping duration (days) by ship mode.

INSIGHT: This chart is really just a sanity check, but it's a reassuring one: shipping duration scales exactly the way you'd expect it to given each mode's name. Same Day is fastest, effectively at 0 days; First Class typically takes 1–3 days; Second Class runs about 2–4 days; and Standard Class is slowest at roughly 4–6 days. What makes this genuinely useful, rather than just a nice-looking chart, is reading it alongside Section 4.3 — since shipping duration showed no meaningful correlation with profit, faster shipping options don't appear to be costing the business profitability. That's a useful, low-risk fact if there's ever a conversation about pushing customers toward faster shipping tiers for a better experience.

4.6 Impact of Discounts on Profit

```
# 6. Discount vs Profitability
plt.figure(figsize=(8, 4))
sns.scatterplot(data=df, x='Discount', y='Profit', hue='Category', alpha=0.7)
plt.axhline(0, color='red', linestyle='--')
plt.title("6. Impact of Discounts on Profit")
plt.tight_layout()
plt.savefig('fig6_discount.png')
plt.show()
```

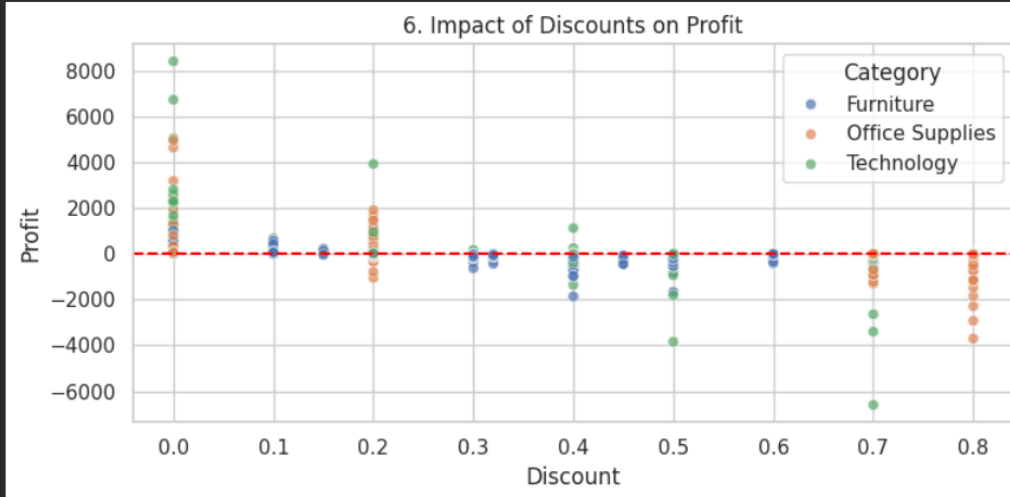


Figure 4.6 — Scatter plot of discount level vs. profit, colored by product category.

INSIGHT: Where Section 4.3 gave us a correlation number, this scatter plot shows exactly what that number looks like in practice. At 0–20% discount, plenty of orders — especially in Technology — are still comfortably profitable, some by several thousand dollars. But starting around the 30–40% discount mark, the picture flips: profit outcomes become overwhelmingly negative across all three categories (Furniture, Office Supplies, and Technology alike), and the worst losses in the entire dataset are concentrated at the 70–80% discount level. Read together with Sections 4.1 and 4.3, this paints a consistent story: discounting isn't inherently bad, but past roughly 30–40%, it stops being a sales tool and starts being a reliable way to lose money. That threshold is a natural, data-backed starting point for any future discussion about a discount approval policy.

4.7 Performance by Customer Segment



Figure 4.7 — Total sales and profit by customer segment.

INSIGHT: Consumer is the largest segment by a comfortable margin, generating well over \$1.1 million in sales, followed by Corporate at roughly \$700,000 and Home Office at around \$430,000. What's worth noticing is that the profit bars shrink in roughly the same proportion as the sales bars across all three segments — none of them stands out as disproportionately more or less profitable relative to its size. That suggests Consumer's lead is mostly a story about order volume rather than about having a fundamentally better margin profile than Corporate or Home Office, which is a useful distinction if the business were ever deciding where to focus growth efforts versus where to focus margin-improvement efforts.

4.8 Top 10 Most and Least Profitable Products

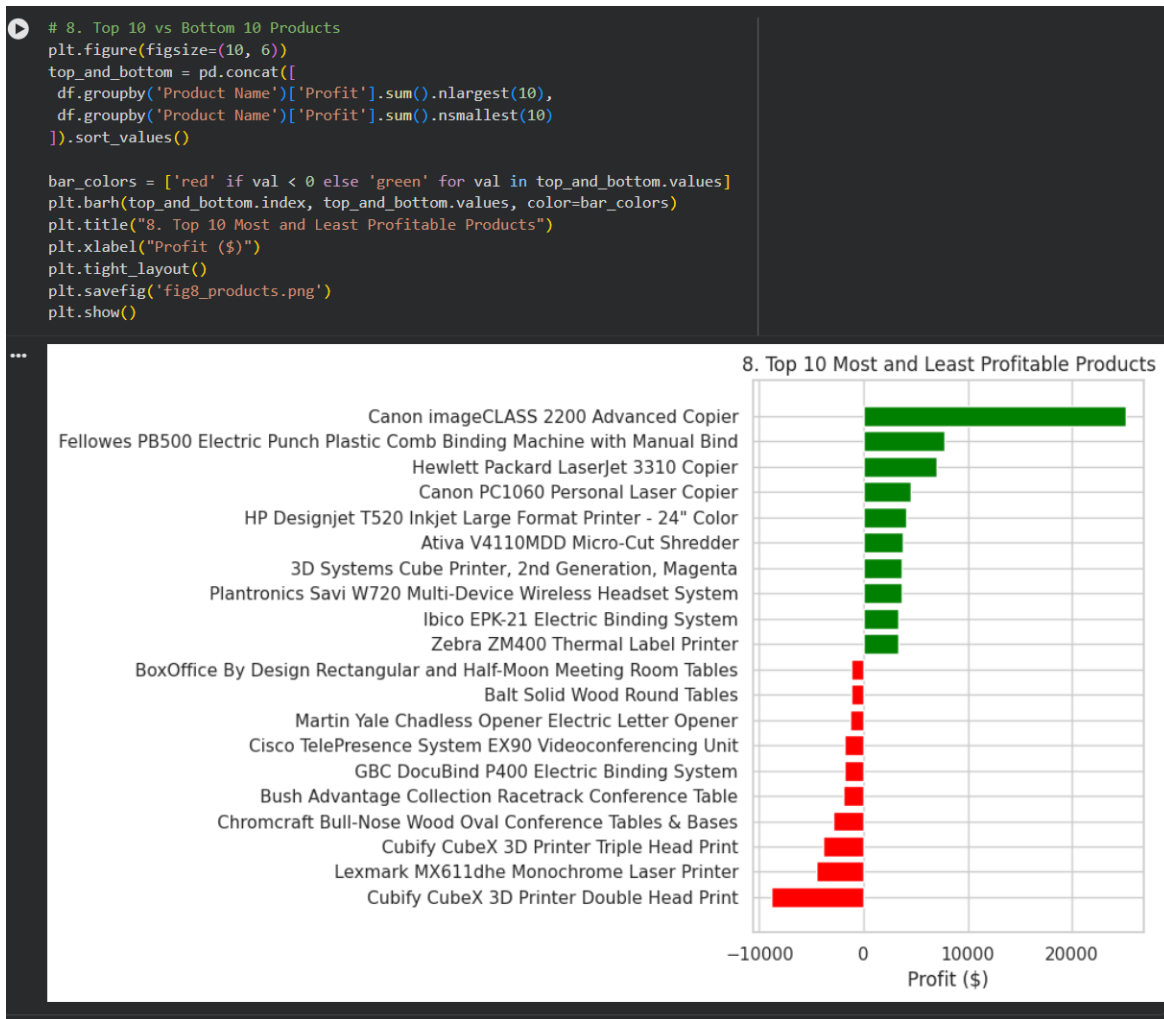


Figure 4.8 — Top 10 most profitable (green) and top 10 least profitable (red) products by total profit.

INSIGHT: Zooming all the way in from sub-categories down to individual products tells its own story. The Canon imageCLASS 2200 Advanced Copier is, by a wide margin, the single most profitable product in the dataset — its bar is noticeably longer than every other product's, on either side of the chart. At the opposite extreme, two variants of the Cubify CubeX 3D Printer (Double Head and Triple Head Print) and the Lexmark MX611dhe Monochrome Laser Printer stand out as the worst performers, each losing several thousand dollars outright. It's also worth noticing how many conference and meeting table products appear among the least profitable items — a direct, product-level confirmation of the Tables sub-category concern first raised in Section 4.2. Taken together, this chart identifies specific, named products that are strong candidates for a pricing review, a renegotiated supplier cost, tighter discount limits, or in the more extreme cases, discontinuation.

5. Key Findings & Recommendations

Pulling everything in Section 4 together, a few themes come up again and again across completely different views of the same data — which is usually a good sign that they're real patterns and not coincidences.

5.1 Key Findings

- Overall profit margin sits at 12.47%, and sales have been growing faster than profit across 2016–2019 — this is a gradually widening gap rather than a one-off dip, which is worth watching going forward.
- Discounting is the single strongest driver of profit loss found anywhere in the data ($r = -0.86$ between discount and profit margin); orders discounted beyond roughly 30–40% are rarely profitable, regardless of category.
- The Tables sub-category and the Central region show up as underperformers across three independent views — the sub-category chart, the regional margin boxplot, and the least-profitable-products list — which is a strong signal rather than a one-off anomaly.
- Shipping speed showed no measurable relationship with profitability, so faster shipping options can likely be offered or promoted without an apparent hit to margin.
- Profitability is concentrated rather than evenly spread: a small number of standout products (led by the Canon imageCLASS 2200 Advanced Copier) contribute a disproportionate share of total profit, while a small number of chronic underperformers (the Cubify CubeX 3D Printers in particular) drive a disproportionate share of the losses.

5.2 Recommendations

- Introduce a discount cap, or at least a tiered approval step, for discounts above roughly 30–40% — particularly in Technology and Office Supplies, where the largest profitable volumes are concentrated.
- Run a focused pricing and cost review on the Tables sub-category and on the Central region's discounting practices, since both show up consistently as problem areas.
- Reassess or consider discontinuing chronically loss-making SKUs such as the Cubify CubeX 3D Printers and the Lexmark MX611dhe, unless there's a clear strategic reason (e.g. drawing in other purchases) to keep them.
- Continue to promote and prioritize high-margin, high-volume products like the Canon imageCLASS 2200 Advanced Copier, and look for what makes it work so well as a possible model for other product lines.

5.3 Tools & Techniques Used

- **pandas** — reading the Excel workbook, cleaning missing/duplicate data, merging the three sheets, and grouping/aggregating for every chart and KPI.

- **numpy** — vectorized conditional logic (`np.where`, `np.select`) used to build the Profit Margin and Sales Performance Category features without slow row-by-row loops.
- **matplotlib & seaborn** — every visualization in this report: line charts, grouped bar charts, a correlation heatmap, boxplots, and a category-colored scatter plot.

6. Limitations & Possible Next Steps

No analysis like this is the final word, and it's worth being upfront about where this one has room to grow. A few honest limitations, and where I'd take this next if I kept going:

- The correlations discussed in Section 4.3, including the strong discount–margin relationship, describe association, not proof of cause and effect. A next step worth doing would be a more formal regression model to estimate the actual dollar impact of a given discount level, holding category and region constant.
- The KPIs and charts summarize the full 2016–2019 period. It would be worth re-running the same analysis on just the most recent year to see whether the patterns found here (like the Central region's weaker margins) are still true today or have already started to improve.
- This report only covers the Orders, Returns, and People sheets. If cost or supplier data were available, it would help explain why specific products like the Cubify CubeX 3D Printers are so consistently unprofitable — is it pricing, unit cost, or something else?
- Turning this analysis into a live, interactive Power BI or Tableau dashboard would let a manager filter by region, category, or date range directly, instead of relying on a fixed set of static charts.